CASE 01 | Dataset loading

SIDE-BY-SIDE: GO TASK / CHATGPT DEVELOPMENT / FINAL GO CODE / EXECUTION EVIDENCE

1 GO TASK / REQUEST

Load the lecturer-provided CSV in Go; verify observation and column counts.

Observed outcome

53 observations; six columns.

2 CHATGPT DEVELOPMENT RECORD

DEVELOPMENT TASK SUMMARY

The exact original ChatGPT prompt is not retained. Task recorded as:

"Load the lecturer-provided CSV in Go; verify observation and column counts."

AI OUTPUT / RESULT

Implemented in main.go. Full original assistant reply unavailable; final code is shown in the adjacent column.

3 FINAL GO SOURCE

main.go | lines 13-35

```
// ================================================
// NGDS ASSIGNMENT 2
// GERMANY CO2 EMISSIONS ANALYSIS
//
// PHASE 1: DATASET LOADING
// PHASE 2: DATA CLEANING AND STATISTICS
// PHASE 3: EXPLORATORY DATA VISUALISATION
// ================================================

// Observation represents one year of CO2 emissions.
type Observation struct {
    Year      int
    Emissions float64
}


func main() {

    fmt.Println("====================================...
    fmt.Println("NGDS ASSIGNMENT 2")
    fmt.Println("GERMANY CO2 EMISSIONS ANALYSIS")
    fmt.Println("====================================...

    // ========================================
```

4 TERMINAL / COPILOT



Original VS Code screenshot

Screenshot 2026-11-10 at 17.21.07.png

EVIDENCE STATUS / INTERPRETATION

Development task summary; exact historical ChatGPT wording not available.

Source code is an extract of the available final Go files. Screenshots are authentic; terminal reruns are from 10 Nov 2026.

CASE 02 | Cleaning and validation

SIDE-BY-SIDE: GO TASK / CHATGPT DEVELOPMENT / FINAL GO CODE / EXECUTION EVIDENCE

1 GO TASK / REQUEST

Check column headers, missing or invalid values, and duplicate years.

Observed outcome

53 valid; zero missing, invalid or duplicate-year records.

2 CHATGPT DEVELOPMENT RECORD

DEVELOPMENT TASK SUMMARY

The exact original ChatGPT prompt is not retained. Task recorded as:

"Check column headers, missing or invalid values, and duplicate years."

AI OUTPUT / RESULT

Implemented in main.go. Full original assistant reply unavailable; final code is shown in the adjacent column.

3 FINAL GO SOURCE

main.go | lines 14-36

```
// NGDS ASSIGNMENT 2
// GERMANY CO2 EMISSIONS ANALYSIS
//
// PHASE 1: DATASET LOADING
// PHASE 2: DATA CLEANING AND STATISTICS
// PHASE 3: EXPLORATORY DATA VISUALISATION
// ================================================


// Observation represents one year of CO2 emissions.
type Observation struct {
    Year      int
    Emissions float64
}


func main() {

    fmt.Println("======================================...
    fmt.Println("NGDS ASSIGNMENT 2")
    fmt.Println("GERMANY CO2 EMISSIONS ANALYSIS")
    fmt.Println("======================================...

    // ========================================
    // PHASE 1: DATASET LOADING
```

4 TERMINAL / COPILOT



Original VS Code screenshot

Screenshot 2026-11-10 at 17.21.11.png

EVIDENCE STATUS / INTERPRETATION

Development task summary; source and execution are authentic.

Source code is an extract of the available final Go files. Screenshots are authentic; terminal reruns are from 10 Nov 2026.

CASE 03 | Descriptive statistics

SIDE-BY-SIDE: GO TASK / CHATGPT DEVELOPMENT / FINAL GO CODE / EXECUTION EVIDENCE

1 GO TASK / REQUEST

Calculate mean, median, standard deviation and extreme CO2 years.

Observed outcome

Mean 934.4943; minimum 642.0878 (2020).

2 CHATGPT DEVELOPMENT RECORD

DEVELOPMENT TASK SUMMARY

The exact original ChatGPT prompt is not retained. Task recorded as:

"Calculate mean, median, standard deviation and extreme CO2 years."

AI OUTPUT / RESULT

Implemented in main.go. Full original assistant reply unavailable; final code is shown in the adjacent column.

3 FINAL GO SOURCE

main.go | lines 183-205

```
    fmt.Println("Duplicate years:", duplicates)

    // ========================================
    // PHASE 2: DESCRIPTIVE STATISTICS
    // ========================================

    fmt.Println("\nDESCRIPTIVE STATISTICS")
    fmt.Println("-----------------------------------...

    n := len(data)
    sum := 0.0

    minObs := data[0]
    maxObs := data[0]

values := make([]float64, 0, n)

// Calculate sum, minimum and maximum
for _, obs := range data {

    sum += obs.Emissions

    values = append(values, obs.Emissions)
```

4 TERMINAL / COPILOT



Original VS Code screenshot

Screenshot 2026-11-10 at 17.21.13.png

EVIDENCE STATUS / INTERPRETATION

Development task summary; terminal output is authentic.

Source code is an extract of the available final Go files. Screenshots are authentic; terminal reruns are from 10 Nov 2026.

CASE 04 | Exploratory analysis

SIDE-BY-SIDE: GO TASK / CHATGPT DEVELOPMENT / FINAL GO CODE / EXECUTION EVIDENCE

1 GO TASK / REQUEST

Create a historical annual CO2 trend plot using Go.

Observed outcome

germany_co2_trend.png generated.

2 CHATGPT DEVELOPMENT RECORD

DEVELOPMENT TASK SUMMARY

The exact original ChatGPT prompt is not retained. Task recorded as:

"Create a historical annual CO2 trend plot using Go."

AI OUTPUT / RESULT

Implemented in eda.go. Full original assistant reply unavailable; final code is shown in the adjacent column.

3 FINAL GO SOURCE

eda.go | lines 12-34

```
func plotEmissions(data []Observation) error {

    p := plot.New()

    p.Title.Text = "Germany CO2 Emissions (1970-2022)"
    p.X.Label.Text = "Year"
    p.Y.Label.Text = "CO2 Emissions (MtCO2e)"

    points := make(plotter.XYs, len(data))

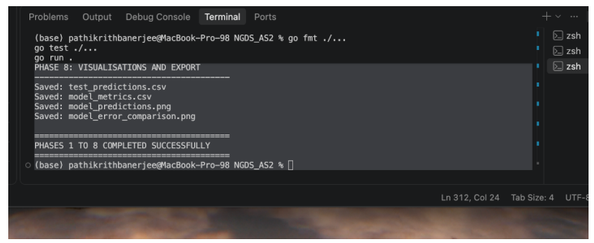
    for i, obs := range data {
        points[i].X = float64(obs.Year)
        points[i].Y = obs.Emissions
    }


line, err := plotter.NewLine(points)
if err != nil {
    return err
}

p.Add(line)

err = p.Save(
```

4 TERMINAL / COPILOT



Original VS Code screenshot

Screenshot 2026-11-10 at 16.57.56.png

EVIDENCE STATUS / INTERPRETATION

Development task summary; screenshot proves generated file exists.

Source code is an extract of the available final Go files. Screenshots are authentic; terminal reruns are from 10 Nov 2026.

CASE 05 | OLS regression

SIDE-BY-SIDE: GO TASK / CHATGPT DEVELOPMENT / FINAL GO CODE / EXECUTION EVIDENCE

1 GO TASK / REQUEST

Fit year-centred OLS on chronological training data; evaluate holdout.

Observed outcome

Holdout MAE 31.2213; RMSE 35.2197; R2 0.6572.

2 CHATGPT DEVELOPMENT RECORD

DEVELOPMENT TASK SUMMARY

The exact original ChatGPT prompt is not retained. Task recorded as:

"Fit year-centred OLS on chronological training data; evaluate holdout."

AI OUTPUT / RESULT

Implemented in regression.go. Full original assistant reply unavailable; final code is shown in the adjacent column.

3 FINAL GO SOURCE

regression.go | lines 21-43

```
func fitLinear(data []Observation) LinearModel {

    if len(data) == 0 {
        return LinearModel{Intercept: 0, Slope: 0}
    }

    if len(data) == 1 {
        return LinearModel{
            Intercept: data[0].Emissions,
            Slope:     0,
        }
    }

    n := float64(len(data))

    sumX := 0.0
    sumY := 0.0

    // Centre years around 1970
    for _, obs := range data {
        sumX += float64(obs.Year - 1970)
        sumY += obs.Emissions
    }
```

4 TERMINAL / COPILOT



Original VS Code screenshot

Screenshot 2026-11-10 at 17.21.16.png

EVIDENCE STATUS / INTERPRETATION

Development task summary; latest uploaded regression.go is used.

Source code is an extract of the available final Go files. Screenshots are authentic; terminal reruns are from 10 Nov 2026.

CASE 06 | Naive baseline

SIDE-BY-SIDE: GO TASK / CHATGPT DEVELOPMENT / FINAL GO CODE / EXECUTION EVIDENCE

1 GO TASK / REQUEST

Predict each holdout year from the final training-year emission.

Observed outcome

Holdout MAE 54.1380; RMSE 75.9480.

2 CHATGPT DEVELOPMENT RECORD

DEVELOPMENT TASK SUMMARY

The exact original ChatGPT prompt is not retained. Task recorded as:

"Predict each holdout year from the final training-year emission."

AI OUTPUT / RESULT

Implemented in baseline.go. Full original assistant reply unavailable; final code is shown in the adjacent column.

3 FINAL GO SOURCE

baseline.go | lines 11-33

```
func runBaseline(data []Observation) {

    split := int(float64(len(data)) * 0.8)

    if split < 1 || split >= len(data) {
        fmt.Println("Insufficient data for baseline ev...
        return
    }

    train := data[:split]
    test := data[split:]

    lastValue := train[len(train)-1].Emissions

    sumAbsError := 0.0
    sumSquaredError := 0.0
    sumActual := 0.0

    for _, obs := range test {
        sumActual += obs.Emissions
    }

    meanActual := sumActual / float64(len(test))
```

4 TERMINAL / COPILOT



Original VS Code screenshot

Screenshot 2026-11-10 at 17.21.18.png

EVIDENCE STATUS / INTERPRETATION

Development task summary; terminal output is authentic.

Source code is an extract of the available final Go files. Screenshots are authentic; terminal reruns are from 10 Nov 2026.

CASE 07 | Quadratic regression

SIDE-BY-SIDE: GO TASK / CHATGPT DEVELOPMENT / FINAL GO CODE / EXECUTION EVIDENCE

1 GO TASK / REQUEST

Fit a quadratic model and compare it with the linear baseline.

Observed outcome

Holdout MAE 74.1423; RMSE 77.7196.

2 CHATGPT DEVELOPMENT RECORD

DEVELOPMENT TASK SUMMARY

The exact original ChatGPT prompt is not retained. Task recorded as:

"Fit a quadratic model and compare it with the linear baseline."

AI OUTPUT / RESULT

Implemented in polynomial.go. Full original assistant reply unavailable; final code is shown in the adjacent column.

3 FINAL GO SOURCE

polynomial.go | lines 21-43

```
func fitQuadratic(data []Observation) (QuadraticModel,...

    n := len(data)

    if n < 3 {
        return QuadraticModel{}, fmt.Errorf(
            "at least 3 observations required",
        )
    }

    X := mat.NewDense(n, 3, nil)
    Y := mat.NewDense(n, 1, nil)

    for i, obs := range data {
        x := float64(obs.Year - 1970)

        X.Set(i, 0, 1)
        X.Set(i, 1, x)
        X.Set(i, 2, x*x)

        Y.Set(i, 0, obs.Emissions)
    }
```

4 TERMINAL / COPILOT



Original VS Code screenshot

Screenshot 2026-11-10 at 17.21.20.png

EVIDENCE STATUS / INTERPRETATION

Development task summary; terminal output is authentic.

Source code is an extract of the available final Go files. Screenshots are authentic; terminal reruns are from 10 Nov 2026.

CASE 08 | Rolling-origin validation

SIDE-BY-SIDE: GO TASK / CHATGPT DEVELOPMENT / FINAL GO CODE / EXECUTION EVIDENCE

1 GO TASK / REQUEST

Use expanding-window one-year-ahead forecasts without future leakage.

Observed outcome

22 folds; naive MAE 22.4240 vs linear 36.8410.

2 CHATGPT DEVELOPMENT RECORD

DEVELOPMENT TASK SUMMARY

The exact original ChatGPT prompt is not retained. Task recorded as:

"Use expanding-window one-year-ahead forecasts without future leakage."

AI OUTPUT / RESULT

Implemented in validation.go. Full original assistant reply unavailable; final code is shown in the adjacent column.

3 FINAL GO SOURCE

validation.go | lines 13-35

```
func validationMetrics(errors []float64) (float64, flo...

    if len(errors) == 0 {
        return math.NaN(), math.NaN()
    }

    sumAbsolute := 0.0
    sumSquared := 0.0

    for _, err := range errors {
        sumAbsolute += math.Abs(err)
        sumSquared += err * err
    }

    n := float64(len(errors))

    mae := sumAbsolute / n
    rmse := math.Sqrt(sumSquared / n)

    return mae, rmse
}

// Run expanding-window one-year-ahead validation.
```

4 TERMINAL / COPILOT



Original VS Code screenshot

Screenshot 2026-11-10 at 17.21.22.png

EVIDENCE STATUS / INTERPRETATION

Development task summary; horizon differs from holdout.

Source code is an extract of the available final Go files. Screenshots are authentic; terminal reruns are from 10 Nov 2026.

CASE 09 | Plots and exports

SIDE-BY-SIDE: GO TASK / CHATGPT DEVELOPMENT / FINAL GO CODE / EXECUTION EVIDENCE

1 GO TASK / REQUEST

Export CSV predictions and metrics; save Go-generated plots.

Observed outcome

Four output files saved; all eight phases completed.

2 CHATGPT DEVELOPMENT RECORD

DEVELOPMENT TASK SUMMARY

The exact original ChatGPT prompt is not retained. Task recorded as:

"Export CSV predictions and metrics; save Go-generated plots."

AI OUTPUT / RESULT

Implemented in results.go. Full original assistant reply unavailable; final code is shown in the adjacent column.

3 FINAL GO SOURCE

results.go | lines 255-277

```
func runResults(data []Observation) error {

    fmt.Println("\nPHASE 8: VISUALISATIONS AND EXPORT")
    fmt.Println("-----------------------------------...

    // Chronological 80/20 split.
    split := int(float64(len(data)) * 0.8)

    if split < 3 || split >= len(data) {
        return fmt.Errorf("insufficient observations")
    }

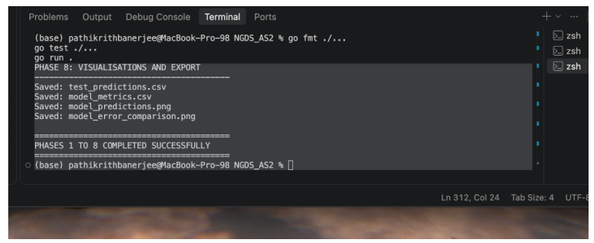
    train := data[:split]
    test := data[split:]

    // Fit linear regression using training data.
    linear := fitLinear(train)

    // Fit quadratic regression using training data.
    quadratic, err := fitQuadratic(train)

    if err != nil {
        return err
```

4 TERMINAL / COPILOT



Original VS Code screenshot

Screenshot 2026-11-10 at 16.57.56.png

EVIDENCE STATUS / INTERPRETATION

Development task summary; latest uploaded results.go is used.

Source code is an extract of the available final Go files. Screenshots are authentic; terminal reruns are from 10 Nov 2026.

CASE 10 | Independent AI code review

SIDE-BY-SIDE: GO TASK / CHATGPT DEVELOPMENT / FINAL GO CODE / EXECUTION EVIDENCE

1 GO TASK / REQUEST

Review OLS, year centring, degenerate inputs, leakage and Go tests; change only genuine problems.

Observed outcome

Copilot revised regression.go and metrics_test.go.

2 CHATGPT DEVELOPMENT RECORD

ACTUAL USER PROMPT (Copilot)

Review OLS, year centring, degenerate inputs, leakage and Go tests; change only genuine problems.

AI response (visible in screenshot): tightened degenerate-input handling; added Go test coverage; reported passing tests.

3 FINAL GO SOURCE

regression.go | lines 21-43

```
func fitLinear(data []Observation) LinearModel {

    if len(data) == 0 {
        return LinearModel{Intercept: 0, Slope: 0}
    }

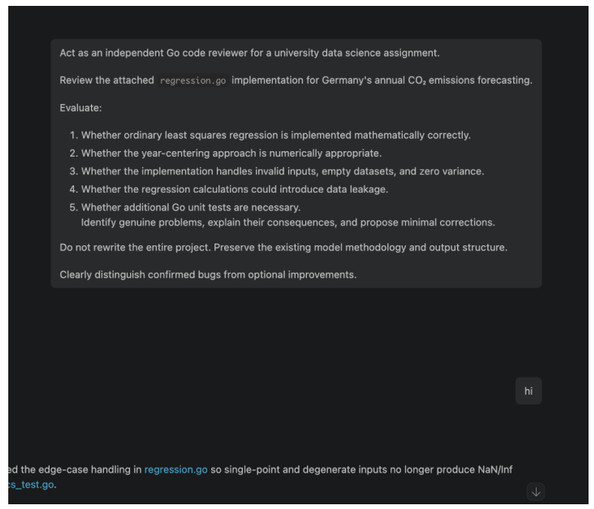
    if len(data) == 1 {
        return LinearModel{
            Intercept: data[0].Emissions,
            Slope:     0,
        }
    }

    n := float64(len(data))

    sumX := 0.0
    sumY := 0.0

    // Centre years around 1970
    for _, obs := range data {
        sumX += float64(obs.Year - 1970)
        sumY += obs.Emissions
    }
```

4 TERMINAL / COPILOT



Original VS Code screenshot

Screenshot 2026-11-10 at 17.16.33.png

EVIDENCE STATUS / INTERPRETATION

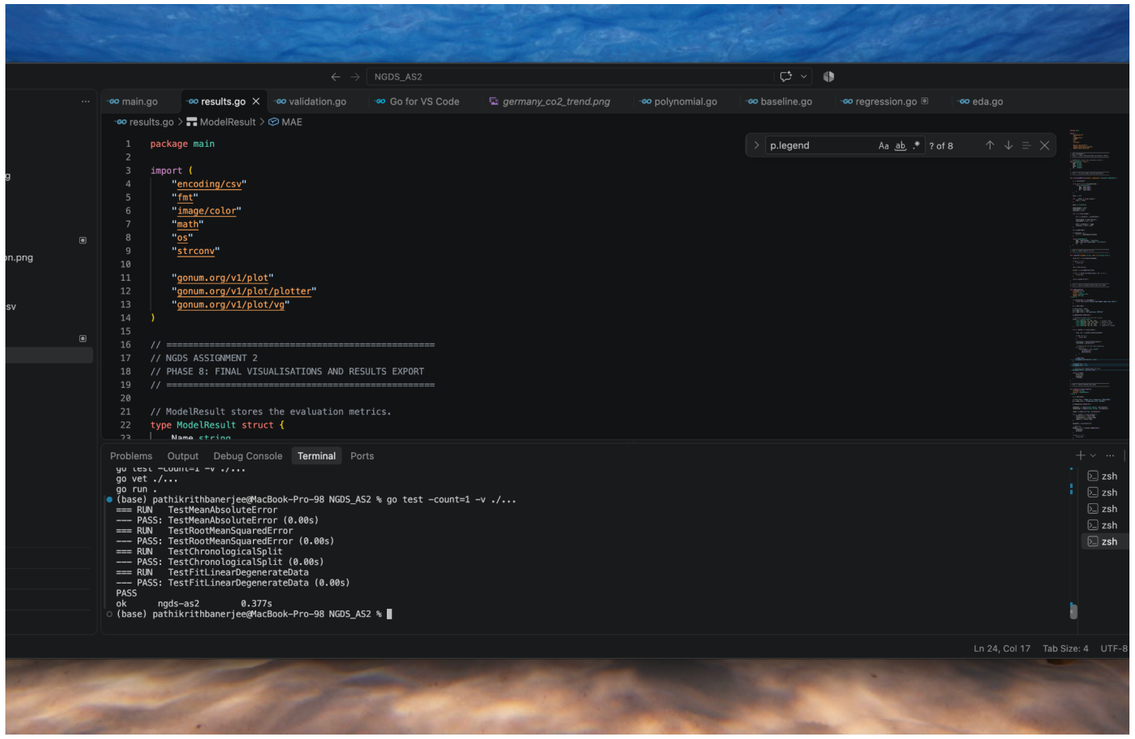
Authentic Copilot prompt screenshot; independent test evidence on next page.

Source code is an extract of the available final Go files. Screenshots are authentic; terminal reruns are from 10 Nov 2026.

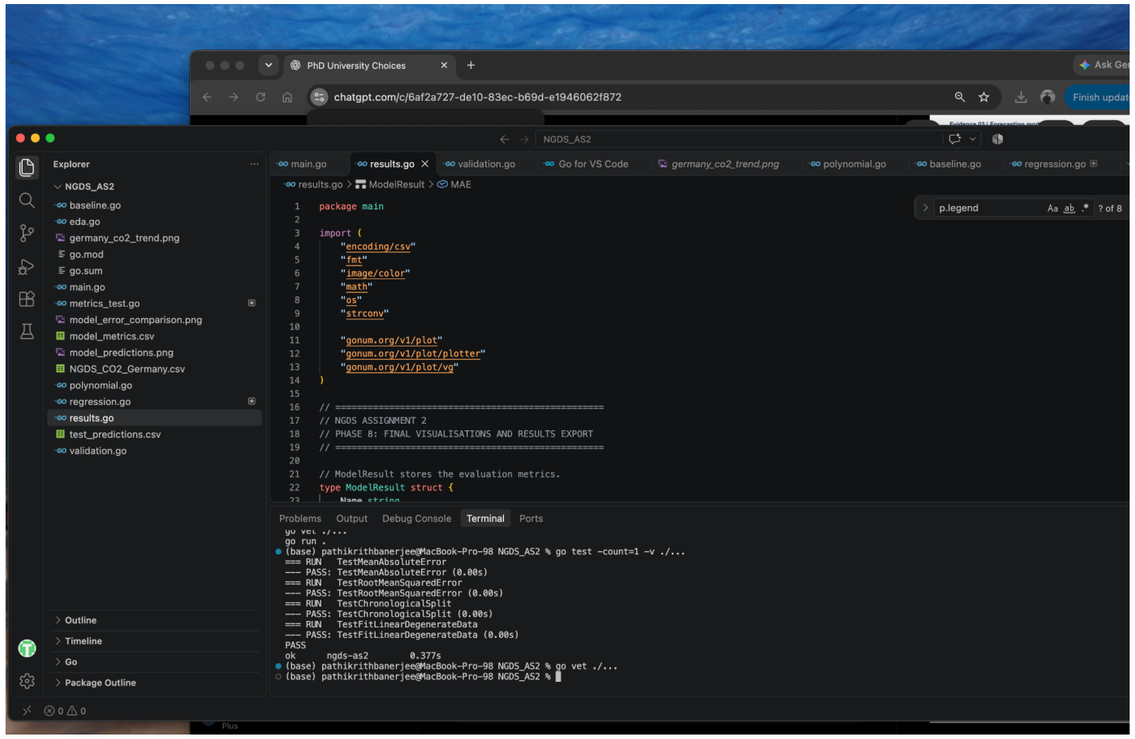
CASE 10 | Independent execution verification

Direct terminal evidence supporting the Copilot code-review interaction

Four Go tests passed



go vet completed without diagnostics



Independent commands: go test -count=1 -v ./... | go vet ./... | go run .

Interpretation: four passing tests and a clean vet run support correctness of the checked behaviour; they do not establish complete test coverage.

Evidence note: the initial 3 November ChatGPT prompts cannot be recovered verbatim from available records. No simulated historical chat screenshots are included.

VERIFICATION ADDENDUM | 10 NOVEMBER 2026

NGDS Assignment 2 — Germany CO2 Emissions Forecasting (Go)

This addendum updates the earlier CASE 10 evidence. The earlier statement of **four passing tests** describes an intermediate project state; the final verified state is **13 passing tests**. Original cases are retained for traceability.

1. Final verification register

Group	Verified tests	Result
Original checks	MAE, RMSE, chronological split, degenerate linear fit	4 PASS
Regression checks	Known coefficients, known predictions, input preservation, constant target, empty input	5 PASS
Validation checks	Known MAE/RMSE, zero errors, empty errors, rolling-window boundaries	4 PASS
Total	Go test run on 10 November 2026	13 PASS

2. Commands and observed outcomes

`go test -count=1 -v ./...` — 13 tests PASS; package ngds-as2, 0.250 s (terminal transcript).
`go vet ./...` and `go build ./...` — completed without reported errors in the supplied terminal run.
`git status` — clean; main up to date with origin/main.

3. Methodological checks

Regression: direct tests check ordinary least-squares coefficients and predictions against known values, plus input preservation and boundary conditions.
Rolling origin: tests check error metrics, a 22-fold window schedule, chronological boundaries and exclusion of the 2012–2022 holdout. The boundary test reproduces the window logic; it does not directly execute every branch of the validation pipeline.

4. Debugging and provenance

Earlier errors were caused by an accidental `EOF` token and placement of a Go test inside the separate `evidence/` package. Moving the test to the project root and correcting the file resolved the errors. Verification changes were committed as `0ed5b4b` and `577fd41` on the main branch.

5. AI-assisted development reflection

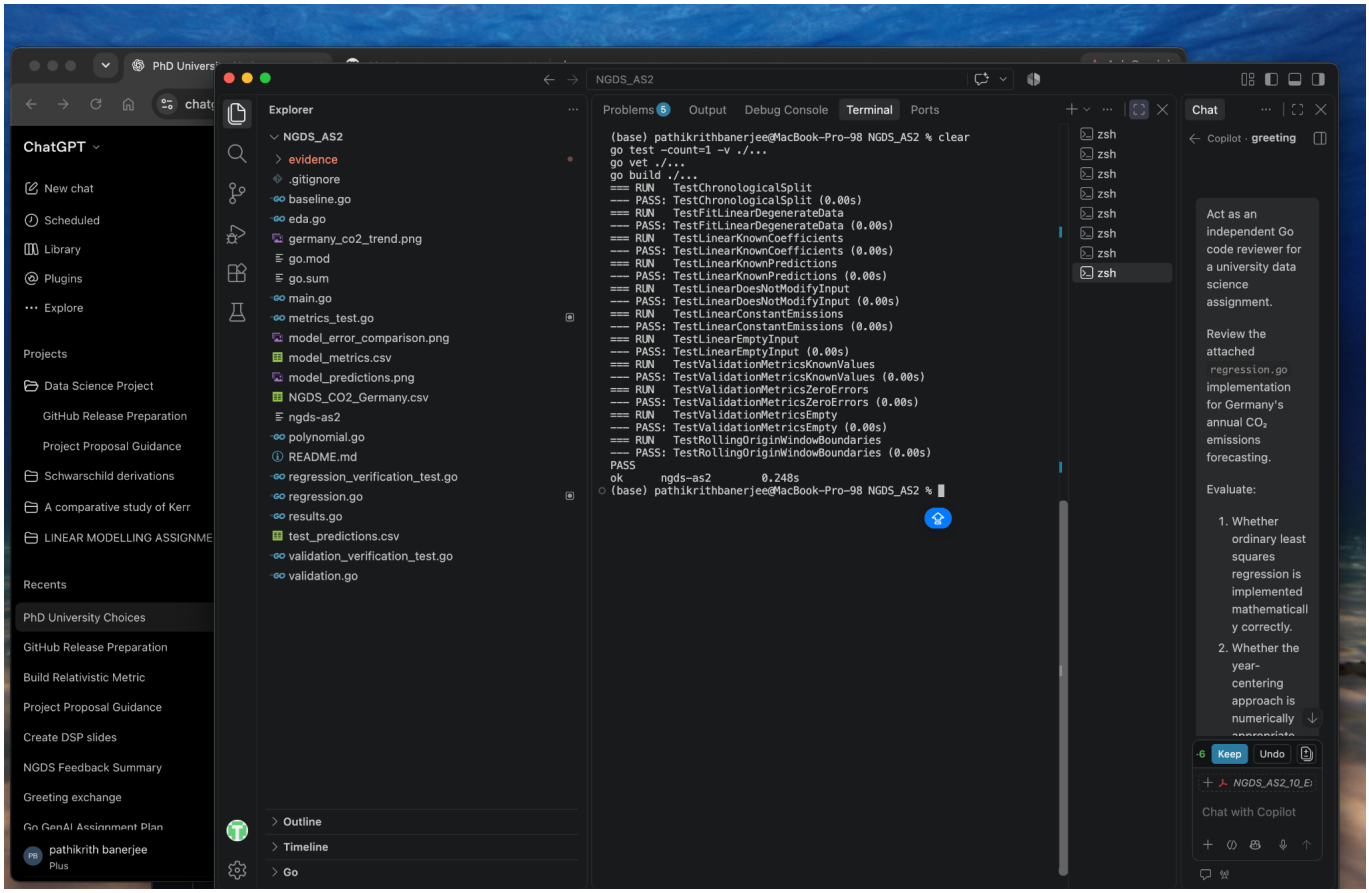
Explain: specify the Go task and forecasting constraints. **Write:** implement and inspect Go source. **Review:** request scrutiny of OLS and temporal leakage. **Test:** execute known-answer and boundary tests, vet and build. **Reflect:** preserve the chronology of errors and fixes, and distinguish verified behaviour from broader claims. This is a retrospective workflow description, not a verbatim historical prompt log.

Limitations: Passing tests do not prove all code paths correct or establish future forecast accuracy. The 22 one-year-ahead validation folds answer a different question from the fixed-origin 11-year test horizon.

Evidence source: supplied terminal transcripts, VS Code screenshots, and GitHub commit output. Original prompts unavailable for some earlier cases; none are invented here.

TERMINAL EVIDENCE | FINAL GO TEST RUN

Authentic VS Code screenshot supplied by the student. The terminal viewport shows the final PASS result and most test names; the complete 13-test transcript is recorded above.



Note: the screenshot also shows an AI code-review panel; this is evidence of the visible review context, not proof of the full underlying conversation or of the AI review quality.